

Analizzatore statico basato sull'interpretazione astratta

UniVR - Dipartimento di Informatica

Alessio Terraroli

2025/2026

Indice

1	Introduzione	2
2	Introduzione	2
3	Background Teorico	2
3.1	Interpretazione Astratta	2
3.2	Soundness e Approssimazione	3
3.3	LUB e Convergenza (Fixpoint)	3
4	Il Dominio dei Segni (Sign Lattice)	3
4.1	Struttura del Reticolo Esteso	3
4.2	Operazioni Aritmetiche Astratte	4

1 Introduzione

L'analisi statica è una tecnica utilizzata per esaminare il codice sorgente di un programma senza eseguirlo. Questa tecnica consente di identificare potenziali errori, vulnerabilità di sicurezza, e problemi di stile nel codice. L'analisi statica può essere eseguita manualmente da sviluppatori esperti o automaticamente utilizzando strumenti software dedicati. Gli strumenti di analisi statica possono essere configurati per rilevare una vasta gamma di problemi, tra cui errori di sintassi, problemi di sicurezza, e violazioni delle convenzioni di codifica. L'analisi statica è una parte importante del processo di sviluppo del software, poiché aiuta a migliorare la qualità del codice e a ridurre il numero di bug e vulnerabilità che possono essere introdotti durante lo sviluppo.

Definizione 1.1. Un **interprete** è un programma che esegue direttamente le istruzioni scritte in un linguaggio di programmazione, senza la necessità di compilare il codice in un formato eseguibile. L'interprete legge il codice sorgente, lo analizza e lo esegue riga per riga o blocco per blocco. Questo processo consente agli sviluppatori di testare e modificare rapidamente il codice, poiché non è necessario attendere la compilazione completa del programma. Gli interpreti sono spesso utilizzati durante la fase di sviluppo e debugging, poiché offrono un feedback immediato sugli errori e sui risultati dell'esecuzione del codice.

2 Introduzione

Il progetto consiste in un interprete astratto sviluppato in Java, progettato per eseguire l'analisi statica di un linguaggio di programmazione. L'architettura si basa sull'uso di **ANTLR4** per l'analisi sintattica, operando direttamente sull'albero sintattico (Parse Tree) tramite il pattern *Visitor*, bypassando rappresentazioni intermedie complesse come AST o CFG. Il framework implementa un *Type System* statico e calcola la semantica astratta per verificare in modo sicuro le proprietà del programma senza eseguirlo.

3 Background Teorico

3.1 Interpretazione Astratta

Introdotta da Patrick e Radhia Cousot nel 1977, l'Interpretazione Astratta fornisce una teoria per l'approssimazione sicura (*sound approximation*) delle strutture matematiche, applicata principalmente alla semantica formale dei programmi. Poiché verificare proprietà semantiche non banali è indecidibile (Teorema di Rice), un interprete astratto sostituisce il dominio infinito dei valori concreti di esecuzione con un **Dominio Astratto** finito, strutturato come reticolo (*Lattice*).

3.2 Soundness e Approssimazione

L'analizzatore calcola uno stato approssimato ma sicuro: garantisce di non assumere mai proprietà false, pur potendo perdere precisione per mantenere la decidibilità. In presenza di incertezza, il sistema degrada verso uno stato di totale ignoranza ($\top$).

3.3 LUB e Convergenza (Fixpoint)

- **Diramazioni Condizionali (If-Else):** Poiché la guardia statica potrebbe valutarsi come $\top$, l'interprete esplora speculativamente entrambi i rami. Le memorie astratte finali vengono riconciliate al *join-point* usando l'operatore matematico **Least Upper Bound (LUB, $\sqcup$)**:

$$\text{Memory}_{\text{final}} = \text{Memory}_{\text{Then}} \sqcup \text{Memory}_{\text{Else}}$$

- **Cicli (While):** Per risolvere staticamente il problema della fermata (*Halting Problem*), l'analisi dei loop esegue una **Fixpoint Computation** (calcolo del punto fisso). L'interprete valuta iterativamente il corpo del ciclo finché la memoria astratta non raggiunge uno stato stazionario:

$$M_{i+1} = M_i \sqcup \text{body}(M_i)$$

Grazie all'altezza finita del reticolo dei segni, la convergenza in $\top$ è matematicamente garantita in pochi millisecondi senza il rischio di loop infiniti.

4 Il Dominio dei Segni (Sign Lattice)

Il dominio concreto degli interi $\mathbb{Z}$ viene astratto nel Dominio dei Segni. Nel progetto, il reticolo base è stato esteso per supportare un'analisi più granulare e precisa, in preparazione all'analisi dei linguaggi dinamici.

4.1 Struttura del Reticolo Esteso

Il reticolo attuale, definito dalla classe `SignType`, supporta le seguenti astrazioni:

- $\perp$ (**Bottom**): Rappresenta uno stato non inizializzato, codice irraggiungibile o un errore statico a runtime (es. divisione per zero).
- **NEG** ($-$): Numeri strettamente negativi.
- **ZERO** (0): Esattamente zero.
- **POS** ($+$): Numeri strettamente positivi.
- **ZERO_PLUS** ($0+$): Numeri maggiori o uguali a zero (zero o positivi).
- **ZERO_MINUS** ($0-$): Numeri minori o uguali a zero (zero o negativi).
- **NOT_ZERO** ($\neq 0$): Numeri positivi o negativi, ma certamente non zero. Generato ad esempio dal LUB tra POS e NEG.
- $\top$ (**Top**): Incertezza totale (potrebbe essere qualsiasi intero).

4.2 Operazioni Aritmetiche Astratte

La classe `SignValue` è responsabile della gestione delle operazioni aritmetiche tra gli elementi del reticolo. Tali operazioni garantiscono l'affidabilità (*soundness*) dell'analisi:

- **Addizione** (`add`): Mantiene la precisione se i segni concordano (es. $POS + POS = POS$, $POS + 0+ = POS$). Se si sommano valori opposti o misti che coprono l'intero asse numerico (es. $POS + NEG$ oppure $NOT_ZERO + 0+$), il risultato degrada in $\top$.
- **Sottrazione** (`sub`): L'operatore meno unario inverte il segno (es. $POS \rightarrow NEG$), permettendo di calcolare la sottrazione tramite l'addizione con l'operando invertito.
- **Moltiplicazione** (`mul`): Calcola accuratamente i prodotti combinati, restituendo valori come $0+$ o $0-$ quando si moltiplicano segni incerti (che includono lo zero) con segni certi.
- **Divisione** (`div`): Gestisce in modo conservativo il troncamento della divisione intera in Java. Ad esempio, la divisione tra POS e POS restituisce $0+$, considerando il caso in cui il numeratore sia minore del denominatore (es. $1/2 = 0$). Se vi è certezza di dividere per zero, viene restituito lo stato di errore $\perp$.